

數位系統設計與實驗 期末專題成果報告

1120339 洪子婕

1. 專題題目

基於 RISC-V Memory-Mapped I/O 與 Data Memory 的多使用者智慧密碼鎖系統。

2. 專題目標與功能

i. 專題目標與改動

原規劃是在 Basys 3 FPGA 上，實作一個小型 RISC-V SoC 多使用者智慧密碼鎖系統，讓 RISC-V CPU 透過 memory-mapped I/O 讀取 Basys 3 上的 switch 與 button，取得使用者 ID 與密碼輸入資料。接著 CPU 會利用 Data Memory 儲存多組使用者密碼、錯誤次數與鎖定狀態，並執行密碼比對、錯誤次數管理與鎖定狀態判斷。輸出部分原規劃使用 LED 顯示 PASS、FAIL、LOCK 狀態，七段顯示器顯示 User ID、錯誤次數或系統狀態，並透過 UART TX，將 USERx PASS、USERx FAIL、USERx LOCK 等事件紀錄傳送至電腦端 Serial Terminal。

最後實際完成的功能：RISC-V CPU 透過 memory-mapped I/O 讀取 SW[9:8] 作為 User ID、讀取 SW[7:0] 作為 8-bit 密碼輸入，以 BTNC 作為確認鍵。CPU 依 User ID 存取 Data Memory 中對應的密碼、錯誤次數與鎖定狀態，完成密碼比對、錯誤次數累計與鎖定功能。密碼正確時，LED0 亮起，七段顯示器顯示 PASS，UART TX 輸出 USERx PASS；當密碼錯誤時，LED1 亮起，七段顯示器顯示 FAIL，UART TX 輸出 USERx FAIL；

當同一使用者連續輸入錯誤密碼達三次時，系統會變鎖定狀態，LED2 亮起，七段顯示器顯示 LOCK，UART TX 輸出 USERx LOCK。此外，不同使用者的錯誤次數與鎖定狀態彼此獨立，因此某一位使用者被鎖定後，其他使用者仍可正常輸入。

與原本規劃相比：

- 原本規劃七段顯示器顯示 User ID、錯誤次數或狀態碼，最後實作時改為顯示 PASS、FAIL、LOCK 狀態文字，修改的原因是讓展示結果更直觀，可以直接從七段顯示器判斷目前系統狀態，不需要再對照狀態碼或錯誤次數。
- 由於七段顯示器本身對部分英文字母的顯示能力有限，因此字母 K 以相似的 H 呈現，但不影響 PASS、FAIL、LOCK 三種狀態的辨識。

ii. 功能

功能	實際完成內容
多使用者驗證	支援 User 0、User 1、User 2、User 3 共四位使用者。
Data Memory 管理	儲存 password[4]、fail_count[4]、lock_state[4]。
錯誤次數管理	每位使用者各自記錄錯誤次數，彼此互不影響。
鎖定機制	同一使用者連續輸錯三次後進入 LOCK 狀態。
LED 顯示	LED0 表示 PASS、LED1 表示 FAIL、LED2 表示 LOCK。
七段顯示器	顯示 PASS、FAIL、LOCK。
UART TX	傳送 USERx PASS、USERx FAIL、USERx LOCK 至電腦端 Serial Terminal。

iii. 使用者操作方式

1. 使用 SW[9:8] 選擇使用者 ID：

00：User 0、01：User 1、10：User 2、11：User 3

2. 使用 SW[7:0] 輸入 8-bit 密碼。

3. 按下 BTNC 作為確認鍵。

4. 若密碼正確：

LED 顯示 PASS 狀態 (LED0 亮)、七段顯示器顯示 PASS、UART TX 傳送

USERx PASS

5. 若密碼錯誤：

LED 顯示 FAIL 狀態 (LED1 亮)、七段顯示器顯示 FAIL、UART TX 傳送

USERx FAIL

6. 若同一使用者錯誤達 3 次：

LED 顯示 LOCK 狀態 (LED2 亮)、七段顯示器顯示 LOCK、UART TX 傳送

USERx LOCK

3. RISC-V 核心必要性說明

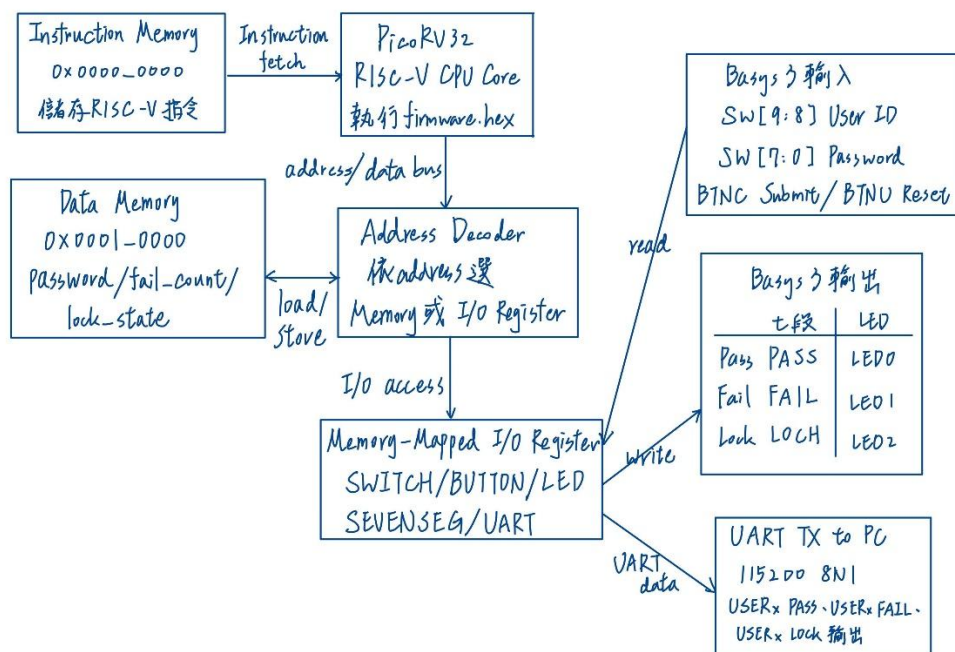
本專題是在 FPGA 上建立一個簡化的 RISC-V SoC，需要 RISC-V CPU 從 Instruction Memory 取出 firmware.hex 所載入的指令並執行，使用 load 指令讀取 SWITCH_REG 與 BUTTON_REG，再用 store 指令寫入 LED_REG、SEVENSEG_REG 與 UART_TX_DATA。Data Memory 則由 CPU 以程式方式管理使用者密碼、錯誤次數與鎖定狀態。

若移除 RISC-V CPU，此專題仍可用純 Verilog FSM 做出密碼鎖，但會失去 CPU 取指令、執行程式、透過 load/store 操作 Data Memory 與 memory-mapped I/O 的意義，因此本專題需要 RISC-V CPU 對記憶體與 FPGA 周邊的程式化控制。

功能	實際完成內容
讀取使用者輸入	CPU 讀取 SWITCH_REG 與 BUTTON_REG。
使用者選擇	CPU 由 SW[9:8] 解出 user_id。
密碼比對	CPU 依 user_id 讀取 password[user_id] 並與 SW[7:0]比較
錯誤次數管理	CPU 更新 fail_count[user_id]。
鎖定狀態管理	CPU 讀寫 lock_state[user_id]。
LED / 七段控制	CPU store 到 LED_REG 與 SEVENSEG_REG。
UART 事件輸出	CPU store ASCII 字元到 UART_TX_DATA。

4. 系統架構圖

系統架構包含 PicoRV32 RISC-V CPU core、instruction memory、data memory、address decoder、memory-mapped I/O registers，以及 Basys 3 的 switch、button、LED、七段顯示器與 UART TX。



5. 硬體設計說明

硬體設計以 Verilog 在 Vivado 中完成，用 Basys 3 實作，主要 Verilog 模組如下：

模組 / 檔案	功能說明
basys3_riscv_lock_top.v	系統 top module，負責整合 PicoRV32 CPU、instruction memory、data memory、address decoder、I/O register、reset extender、LED、七段顯示器與 UART TX。
picorv32.v	開源 RISC-V CPU core，本專題不修改 CPU core 本體，只將其整合到 SoC 架構中。
uart_tx.v	UART TX 底層傳送模組，設定 115200 baud、8 data bits、no parity、1 stop bit，負責將 CPU 寫入的 ASCII byte 轉為 serial output。
uart_event_sender.v	UART 事件字串輸出模組，當 CPU 寫入事件代碼 P、F 或 L 時，此模組會依照目前 User ID 轉換並輸出 USERx PASS、USERx FAIL 或 USERx LOCK 至電腦端 Serial Terminal。
sevensseg_status.v	七段顯示器控制模組，將 CPU 寫入的狀態碼顯示為 PASS、FAIL、LOCK
firmware.hex	RISC-V 控制程式的十六進位機器碼，透過 \$readmemh 載入 Instruction Memory。CPU 執行後會完成密碼比對、錯誤次數管理、鎖定狀態判斷，並將結果寫入 LED、七段顯示器與 UART 相關暫存器。
basys3_riscv_lock.xdc	Basys 3 腳位檔案，包含 clock、switch、button、LED、七段顯示器與 UART TX 腳位。

我自行完成的部分主要包含 SoC 整合、memory map 規劃、I/O register 設計、address decoder 連接、reset 控制、UART TX 整合、七段顯示器狀態設計與實機測試。

6. RISC-V 程式說明

本專題的 RISC-V 程式採用 polling 方式執行，未使用中斷處理。系統啟動後，CPU 會先進行初始化，包含設定初始輸出狀態、錯誤次數與鎖定狀態，並依照 firmware.hex 初始化各使用者的密碼。密碼、fail_count 與 lock_state 由 Data Memory 管理，用來記錄不同使用者的驗證狀態。

初始化完成後，程式會進入主迴圈。CPU 先透過 memory-mapped I/O 持續讀取 BUTTON_REG。按下 BTNC 確認鍵後，CPU 會讀取 SWITCH_REG，從 SW[9:8] 取得 user_id，從 SW[7:0] 取得輸入的 8-bit 密碼。接著 CPU 會根據 User ID 讀取 Data Memory 中對應的密碼、錯誤次數與鎖定狀態。

若該使用者已被鎖定，CPU 會直接輸出 LOCK 狀態，寫入 SEVENSEG_REG 的狀態碼為 9999，LED2 亮起，UART_TX_DATA 的寫入事件碼 L。若該使用者尚未被鎖定，CPU 會將輸入密碼與 Data Memory 中儲存的正確密碼進行比對。若密碼正確，系統會輸出 PASS 狀態，寫入 SEVENSEG_REG 的狀態碼為 1111，LED0 亮起，UART_TX_DATA 寫入事件碼 P。若密碼錯誤，CPU 會將該使用者的 fail_count[user_id] 加一，並判斷錯誤次數是否達三次。

若未達三次，CPU 會輸出 FAIL 狀態，寫入 SEVENSEG_REG 的狀態碼為 2222，LED1 亮起，UART_TX_DATA 寫入事件碼 F；若錯誤次數達三次，CPU 會將 lock_state[user_id] 設為 1，進入 LOCK 狀態，輸出 LOCK 對應的狀態碼與事件碼。

RISC-V firmware 主要負責輸出內部狀態碼與事件碼，七段顯示器顯示的 PASS、FAIL、LOCK，是由 `sevensseg_status.v` 將 `SEVENSEG_REG` 中的 1111、2222、9999 轉換而來；電腦端 Serial Terminal 顯示的 USERx PASS、USERx FAIL、USERx LOCK，則是由 `uart_event_sender.v` 將 `UART_TX_DATA` 中的 P、F、L 事件碼轉換成完整字串。

完成一次判斷與輸出後，程式會回到主迴圈，繼續等待下一次 BTNC 確認輸入。

RISC-V 程式的主要流程如下：

1. 初始化 Data Memory 與輸出狀態
2. 進入 polling 主迴圈
3. 讀取 `BUTTON_REG`，等待 BTNC
4. 讀取 `SWITCH_REG`，取得 `user_id` 與 輸入密碼
5. 根據 User ID 讀取 Data Memory 中對應的 `password`、`fail_count` 與 `lock_state`。
6. 若 `lock_state` 為 1，輸出 LOCK 狀態。
7. 若未鎖定，進行密碼比對。
8. 密碼正確時輸出 PASS 狀態。
9. 密碼錯誤時更新 `fail_count`，並輸出 FAIL 狀態。
10. 若 `fail_count` 達三次，更新 `lock_state`，並輸出 LOCK 狀態。
11. 將結果寫入 `LED_REG`、`SEVENSEG_REG` 與 `UART_TX_DATA`。
12. 由 `sevensseg_status.v` 將狀態碼轉換為七段顯示文字，並由

uart_event_sender.v 將事件碼轉換為完整 UART 字串。

13. 回到主迴圈

7. 開發板與 I/O 配置

本專題使用 Basys 3 作為開發板，輸入端使用 switch 與 button，輸出端使用 LED、七段顯示器與 UART TX。

I/O	用途	說明
SW[7:0]	密碼輸入	輸入 8-bit 密碼。
SW[9:8]	使用者選擇	00=User 0，01=User 1，10=User 2，11=User 3。
BTNC	確認鍵	送出目前 switch 上的 user_id 與 password。
BTNU	Reset	重設系統，重新啟動 CPU。
LED[0]	PASS	密碼正確時亮起。
LED[1]	FAIL	密碼錯誤但尚未鎖定時亮起。
LED[2]	LOCK	該使用者錯三次被鎖定時亮起。
七段顯示器	狀態文字	PASS、FAIL、LOCK、0000=初始。
UART TX	事件輸出	USERx PASS、USERx FAIL、USERx LOCK 115200 baud、8N1。

User	SW[9:8]	預設密碼 SW[7:0]	十六進位
User 0	00	00010010	0x12
User 1	01	00110100	0x34
User 2	10	01010110	0x56
User 3	11	01111000	0x78

8. 實作與測試結果

測試內容包含正確密碼驗證、錯誤密碼判斷、錯誤三次鎖定、多使用者獨立性。

測試項目	輸入	預期輸出	實際結果	是否通過
User 0 正確密碼驗證	SW[9:8]=00 SW[7:0]=00010010 按下 BTNC	七段顯示器顯示 PASS， LED0 亮起， UART TX 輸出 USER0 PASS	顯示 PASS，LED0 亮起，Serial Terminal 顯示 USER0 PASS	是
User 0 錯誤密碼驗證	SW[9:8]=00 SW[7:0]輸入非 00010010 按下 BTNC	七段顯示器顯示 FAIL， LED1 亮起， UART TX 輸出 USER0 FAIL	顯示 FAIL，LED1 亮起，Serial Terminal 顯示 USER0 FAIL	是
同一使用者錯三次鎖定	User 0 連續輸入錯誤密碼三次，每次均按下 BTNC	第三次錯，七段顯示器顯示 LOCK， LED2 亮起， UART TX 輸出 USER0 LOCK	第三次錯顯示 LOCK，LED2 亮起，Serial Terminal 顯示 USER0 LOCK	是
鎖定後不可再登入	User 0 已鎖定後，再輸入 User 0 正確密碼(00010010)並按下 BTNC	系統仍維持 LOCK 狀態， 七段顯示器顯示 LOCK， LED2 亮起，	顯示 LOCK，LED2 亮起，UART TX 輸出 USER0 LOCK，仍維持 LOCK 狀態	是

		UART TX 輸出 USER0 LOCK		
多使用者 狀態獨立 性	User 0 鎖定後， 切換至 User 1， 輸入 User 1 正確 密碼(00110100)， 按下 BTNC	User1 不受 User 0 影 響，七段顯示器顯示 PASS， LED0 亮起， UART TX 輸出 USER1 PASS	User 1 顯示 PASS， LED0 亮起，Serial Terminal 顯示 USER1 PASS	是
User 1 正 確密碼驗 證	SW[9:8]=01 SW[7:0]=00110100 按下 BTNC	七段顯示器顯示 PASS， LED0 亮起， UART TX 輸出 USER1 PASS	顯示 PASS，LED0 亮 起，Serial Terminal 顯 示 USER1 PASS	是
User 2 正 確密碼驗 證	SW[9:8]=10 SW[7:0]=01010110 按下 BTNC	七段顯示器顯示 PASS， LED0 亮起， UART TX 輸出 USER2 PASS	顯示 PASS，LED0 亮 起，Serial Terminal 顯 示 USER2 PASS	是
User 3 正 確密碼驗 證	SW[9:8]=11 SW[7:0]=01111000 按下 BTNC	七段顯示器顯示 PASS， LED0 亮起， UART TX 輸出 USER3 PASS	顯示 PASS，LED0 亮 起，Serial Terminal 顯 示 USER3 PASS	是

9. 成果展示說明

展示前需先在 Vivado 中將 basys3_riscv_lock_top.v 設為 top module，載入 firmware.hex，產生 bitstream 後燒錄到 Basys 3。使用 Tera Term 或其他 serial terminal 設定為 115200 baud、8 data bits、no parity、1 stop bit、flow control none，用來接收 UART TX 輸出。

操作方式如下：上電後或燒錄後，按 BTNU 進行 reset，使 CPU 從 instruction memory 重新執行 firmware.hex。SW[9:8] 選擇 User ID：00、01、10、11 分別代表 User 0 到 User 3，SW[7:0] 輸入 8-bit 密碼，按下 BTNC 送出密碼，CPU 會透過 memory-mapped I/O 讀取輸入並執行判斷。若密碼正確，LED0 亮、七段顯示 PASS，Tera Term 顯示 USERx PASS。若密碼錯誤，LED1 亮、七段顯示 FAIL，Tera Term 顯示 USERx FAIL。若同一 User 錯誤三次，LED2 亮、七段顯示 LOCK，Tera Term 顯示 USERx LOCK，此 User 後續即使輸入正確密碼仍維持 LOCK。

展示影片連結：<https://youtube.com/shorts/7CLQSgbJTwo?si=Gn-RL9X91FDJFabj>

10. 問題、除錯與解決方式

- firmware.hex 無法被讀取:

問題：因 Windows 預設隱藏副檔名，最初的檔案實際名稱為 firmware.hex.txt，導致 Vivado 在執行 \$readmemh 時無法正確找到 firmware.hex

解決方式：將.txt 刪去後就能正常運行。

- Reset 行為不穩定

問題：早期版本按 BTNU 後有時候會 reset，有時沒反應。

解決方式 :加入 reset extender / reset synchronizer，使 BTNU 放開後等一段時間再讓 CPU run。

- 不知道 CPU 是否真的讀到 switch:

問題 :一開始不確定 LED 亮，是否只是 Verilog 直接接線。

解決方式 :先用 firmware 讓 CPU 讀 SWITCH_REG 再寫 LED_REG，確認是

SW -> CPU -> LED 。

11. GitHub 連結

https://github.com/jieisy/riscv_password_lock_basys3

12. 外部來源與開源專案使用說明

來源	用途		本專題修改情形
PicoRV32 / YosysHQ	作為 RISC-V CPU core 。	Github 連結	未修改 CPU core 本體，只進行 SoC 整合。
Digilent Basys 3 Reference Manual	確認 Basys 3 開發板 I/O、 clock、 switch、 button、LED、 七段顯示器與 UART 腳位。	連結	依板子腳位撰寫 XDC 。
Tera Term	作為 PC serial terminal，接收 UART TX 。	Github 連結	用於實機驗證 UART 輸出。

13. AI 協作紀錄摘要

使用 AI 的地方	AI 提供的協助	我如何檢查與修改
題目與架構規劃	協助比較不同專題風險	依課程規範保留 CPU 、 instruction memory 、 data memory 、 I/O register 與周邊。
RISC-V 必要性說明	協助整理 CPU load/store 、 Data Memory 管理與 UART polling 的說明。	確認每個功能確實由 firmware 控制，而不是 直接 Verilog 接線。
Vivado 除錯	協助分析 \$readmemh 路徑、reset 不穩、 button mapping 與 UART 卡住問題。	實測與檢查 LED 、 SW 、 BTNC 、密碼鎖、UART A 、UART P/F/L 事件碼 測試，以及最終 USERx PASS/FAIL/LOCK 字串 輸出測試。
報告整理	協助把規劃書與實作結 果整理成成果報告。	根據實際完成結果修正 內容

14. 個人貢獻說明

本專題為個人專題，主要貢獻如下：

1. 規劃 RISC-V SoC 系統架構與 memory map 。
2. 在 Vivado 中建立 Basys 3 FPGA 專案，加入 Verilog 模組與 XDC 腳位約束。
3. 整合 PicoRV32 CPU core 、 instruction memory 、 data memory 與

memory-mapped I/O。

4. 完成 switch、button、LED、七段顯示器與 UART TX 的硬體連接。
5. 載入、整合並測試 RISC-V firmware.hex，使 CPU 能執行密碼比對與狀態管理。
6. 完成多使用者 password、fail_count、lock_state 管理。
7. 完成錯誤三次鎖定、鎖定後不能登入，以及不同使用者彼此獨立的測試。
8. 完成 UART TX 輸出，並以 Tera Term 驗證。
9. 整理成果報告、測試案例與展示流程。

15. 未完成項目與未來改善

本專題已完成 RISC-V 多使用者密碼鎖的核心功能，包含密碼驗證、錯誤次數管理、使用者鎖定、LED 狀態顯示、七段顯示器狀態文字顯示，以及 UART TX 完整事件輸出。仍有以下可改善方向：

目前狀態	未來改善方式
目前可透過 UART TX 輸出 USERx PASS、USERx FAIL、USERx LOCK 等事件訊息。	未來可加入 UART FIFO 或更完整的傳送緩衝機制，使多筆事件連續輸出時更加穩定。
目前僅完成 UART TX 輸出，尚未支援從電腦端輸入指令。	未來可加入 UART RX，讓管理者透過 Serial Terminal 修改使用者密碼、查詢鎖定狀態或解除鎖定。
目前採用 polling 方式讀取 BTNC 與 UART 狀態。	未來可加入 button interrupt 或 UART interrupt，降低 CPU 長時間輪詢等待的情況。
本次主要以 Basys 3 實機測試完成驗證。	未來可補上完整 testbench 與

	waveform，驗證 address decoder、Data Memory、I/O register 與 UART 傳送行為。
目前密碼為預設固定值，尚未加入加密或動態修改機制。	未來可加入密碼修改功能或簡易加密儲存方式，提高系統安全性。